

Treat the following questions:**Exercise 1: (11 pts)**

Consider the following class named *Car*:

```
public class Car {
    private String brand;
    private int year;
    private String country;

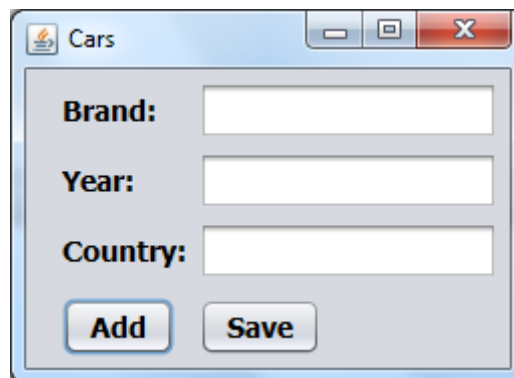
    public Car(String brand, int year, String country) {
        this.brand = brand;
        this.year = year;
        this.country = country;
    }

    public int getYear() {
        return year;
    }

    public String getCountry() {
        return country;
    }

    public String getBrand() {
        return brand;
    }
}
```

Consider the following graphical interface that allows the data input of a *Car* using text fields. The button *Add* allows the creation of a new *Car* object and adds it to an *ArrayList*. The button *Save* allows the saving the *Car* objects of the *ArrayList* to a file. The file's data is loaded at program start-up using the method *load* below.



The corresponding code of this graphical interface is as follows:

```
public class CarFrame extends Frame {
    private TextField brand = new TextField(10);
    private TextField year = new TextField(10);
    private TextField country = new TextField(10);
    private Label lBrand = new Label("Brand:");
    private Label lYear = new Label("Year:");
    private Label lCountry = new Label("Country:");
    private Button add = new Button("Add");
    private Button save = new Button("Save");
    private ArrayList<Car> carsList = new ArrayList<Car>();

    public CarFrame() {
        // to complete ... (1)
        this.pack();
        load();
        // to complete ... (2)
    }

    private void addPerformed() {
        // to complete ... (3)
    }

    private void savePerformed() {
        // to complete ... (4)
    }
}
```

```

    public void save2Performed() {
        // to complete ... (5)
    }

    private void load() {
        // to complete ... (6)
    }
}

```

- Give in (1) the necessary code in order to obtain the above graphical interface. (You may use the layout managers *BorderLayout*, *FlowLayout* and *GridLayout*).
- Give in (2) the code that adds an *ActionListener* to the button *Add* and executes the method *addPerformed*.
- Give in (3) the code that allows the data input in the text fields, create a *Car* object and to add it to the object *carsList*. The text fields should be empty after the addition.
- Give in (4) the code that opens the file « car.txt » and writes in it the content of the *carsList*. Each line must contain the data of one *Car* object as follows:
Mercedes,1990,Germany
- Give in (5) the code that serializes (persists) the *Car* objects of the *carsList* to a file « data.bin ». Which change should you do to the *Car* class regarding the persistence?
- Give in (6) the code that loads the content of the file « car.txt », creates the *Car* objects (one by line) and adds them to the *carsList*.
- The method *parseInt* raises an exception of type *NumberFormatException* if the String passed as parameter cannot be converted to an integer number. Change the code of the method *addPerformed()* in order to support this exception. The raise of the exception will display a *JOptionPane* as follows:



Exercise 2: (5 pts)

- Write the code that draws a rectangle in green and fills it in red.
The rectangle's dimensions are entered in the html code.
- Modify the previous code in order to display an error message if the rectangle's parameters are greater than the applet's dimensions.

Exercise 3: (4 pts)

Suppose that we have a database called « customerDB » with a username « tripoli » and a password « saida ».

This database contains the table « Customer » that is composed of two attributes: « id » and « name » with the following tuples (records):

id	name
10	Sky Net
20	Galaxy
30	Bon Jus
40	Pro Cars

- Open a connection to the database, read the table's data, and put them in an object of type *HashMap*.
- Use an iterator on the *HashMap* already created and then display each tuple on a line.